GOSIM

GOSIM Shenzhen 2026

Agent Centric Engineering

从个人生产力到组织能力

董鑫 · 华为2012基础软件院 项目群总监



世界已经形成四大 AI 圈层

84%

被遗忘的群体

全球化数字鸿沟仍在，广大欠发达地区和人群尚未有效利用 AI 核心能力。

16%

浅表应用

多数用户停留在搜索、基础写作等日常任务，未触及生产力重塑。

0.3%

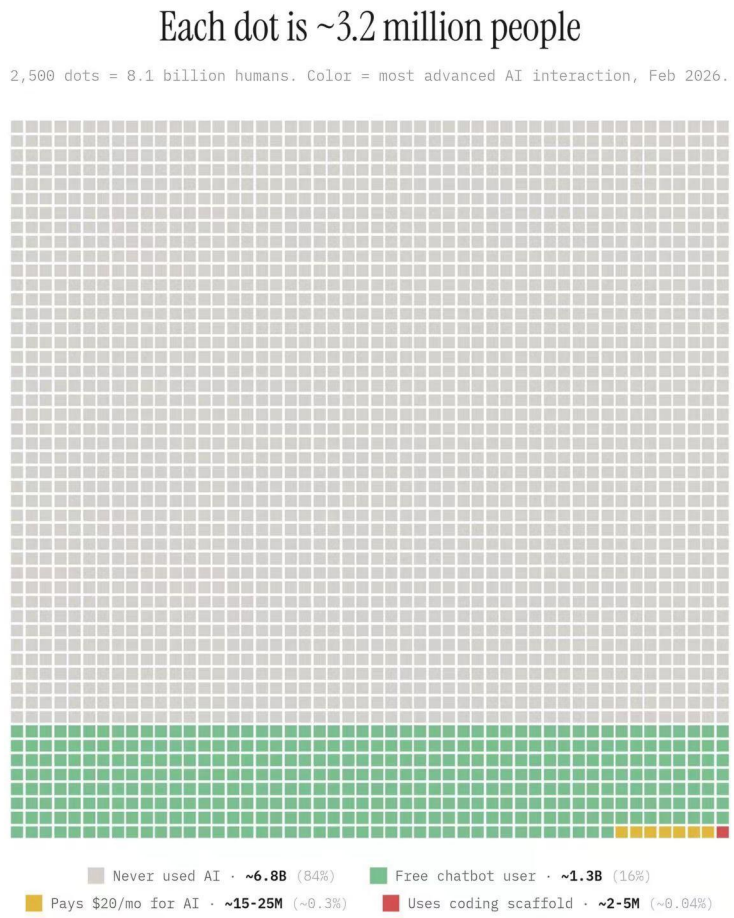
付费壁垒

高阶模型、Prompt 技巧与工作流意识，让精英用户率先突破效率边界。

0.04%

规则制定者

利用 Agent、多模型协作和全栈能力，重构行业工作流的少数人。



原图保留：全球 AI 使用分布示意

从团队尝鲜，到架构、组织与流程重构

01

25.05

团队尝鲜 AI

技术分享索引项目

分享 AI 知识，激发尝试

02

25.07

组建 AI 先锋

每个 PL 团队投入 1–2 人

选拔骨干，组建种子队伍

03

25.09

场景实战

AI 场景实战尝到甜头

项目落地，体验成效

04

26.01

复制经验

团队内复制成功经验

成果分享，规模化应用

05

26.12

体系重塑

架构、组织、流程重构

体系重塑，提升效率

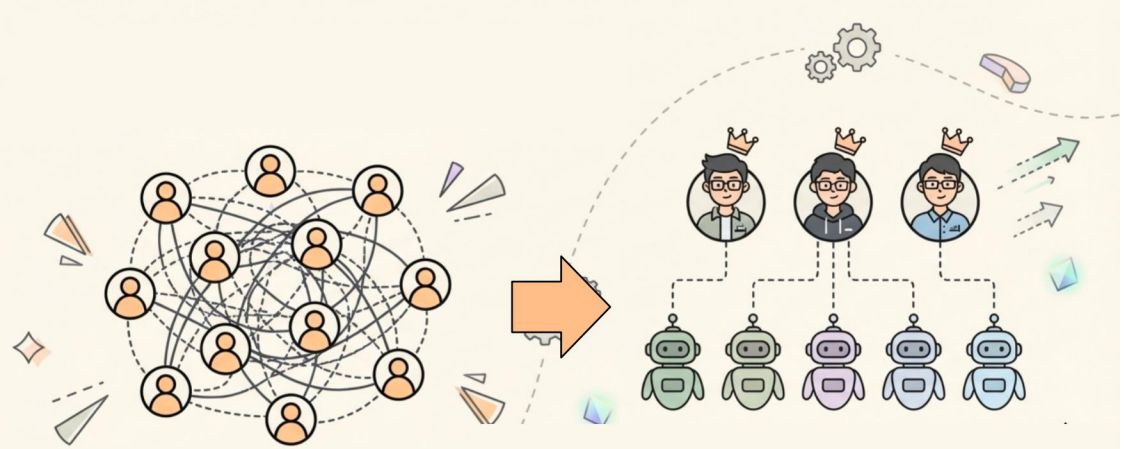
个人级任务效率提升数据

任务	团队	效率提升
三方库开发效率中的实践	三方库团队	产能提升 2.7X-4.5X 1.1 Kloc/人月 → 3-5 Kloc/人月
编译器 Syntax 库适配	编译器团队	60% 5 人天 → 2 人天
编译器 CHIR 插件实现	编译器团队	25% 20 人天 → 15 人天
编译器性能劣化点引入分析	编译器团队	75% 4 人时 → 1 人时
AI 提升工具开发效率实践	工具团队	95% 10 人天 → 0.5 人天
API 测试用 AI 例生成中的实践	API 团队	80% 5 人天 → 1 人天
AI 在 C++ Crash 类稳定性问题分析定位中的经验分享	API 团队	92% 3 人天 → 0.25 人天
AI 辅助编译器前端代码开发	编译器团队	40% 5 人天 → 3 人天
使用 AI 辅助开发者文档英文化的实践	API + 资料团队	90% 40 人天 → 4 人天
AI 在资料整改中的效率提升实践	API 团队	28% 54 人天 → 39 人天
AI 在 Cangjie UI 1.2 项目中的效率提升实践	鸿蒙 UI 团队	33%-80% 3-5 人天 → 1-2 人天
Rules+MCP 工作流对团队新人上手效率的提升	鸿蒙 UI 团队	33%-80% 3-5 人天 → 1-2 人天
Rules+MCP 对团队新人首次独立提交时间的加速	鸿蒙 UI 团队	50% 5 人天 → 2.5 人天
Rule+MCP 对 Code Review 效率的提升实践	鸿蒙 UI 团队	60%-100% 5-8 次/PR → 0-2 次/PR
使用 AI 开发代码仓质量看护看板系统的实践	工具团队	80%-86% 10-14 人天 → 2 人天

15 项个人级任务实践；保留原始产能、耗时及 PR 指标口径

个人效率提升 ≠ 团队效率提升

现实　　理想



1/5　编码开发时间只占项目一小部分

大型项目中真正写代码的时间只占 1/5–1/3，其余都是非编码工作。

沟通　大量时间花在沟通对齐上

需求澄清、方案对齐、跨团队协作、评审反馈，占据工程师大部分日常。

E2E　真正的研发端到端流程

需求分析、对齐、实现、验证、修复，每个环节都决定交付速度。

系统性能提升的第一性原理：阿姆达尔定律

系统加速比

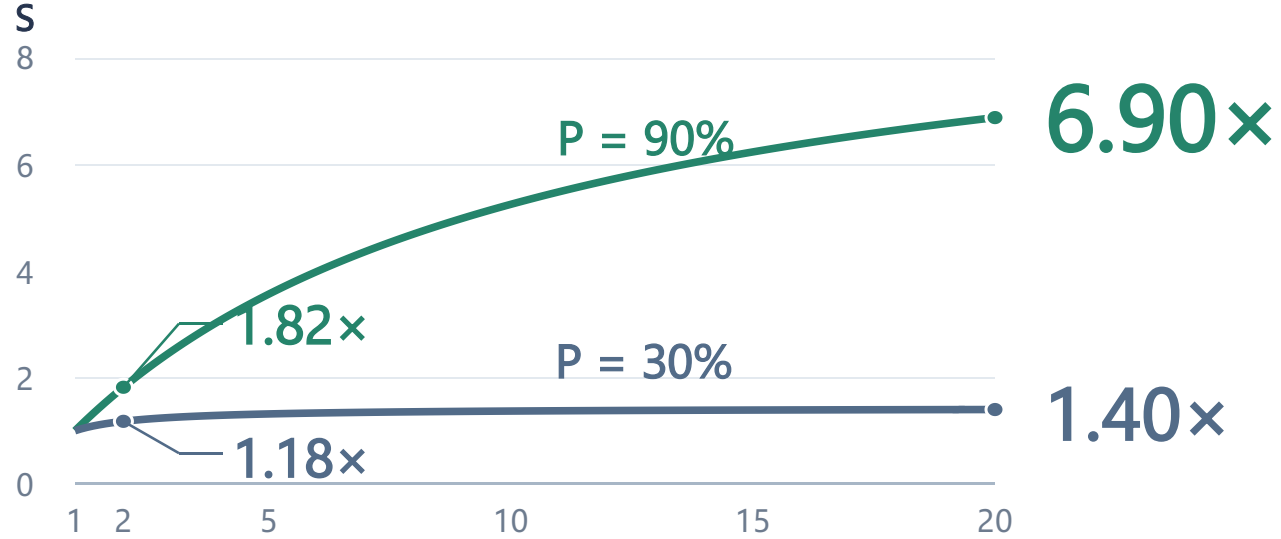
$$S = \frac{1}{(1 - P) + P/N}$$

Amdahl’s Law

P 可并行部分耗时占比

N 理想并行度

并行度的收益，取决于可并行的比例



理想并行度 N

固定工作量的理想模型，忽略协同开销

AI提效的启示

P AI 参与度

Agent Centric Engineering (ACE)

扩大可被 AI 加速的工作范围

N 系统并行度

Ability-based Task Matching (ATM)

提升有效并行执行能力

提升AI参与度（P) - Agent Centric Engineering

SYSTEM CONTROL

Human On the Loop

系统可控

目标 边界 | 资源 门禁 | 节奏 沉淀

可控：定规则、定边界、定节奏。

CONTINUOUS AGENT LOOP



规则 | 执行 | 沉淀

RESULT TRUST

Human In the Loop

结果可信

风险 识别 | 人工 裁决 | 质量 背书

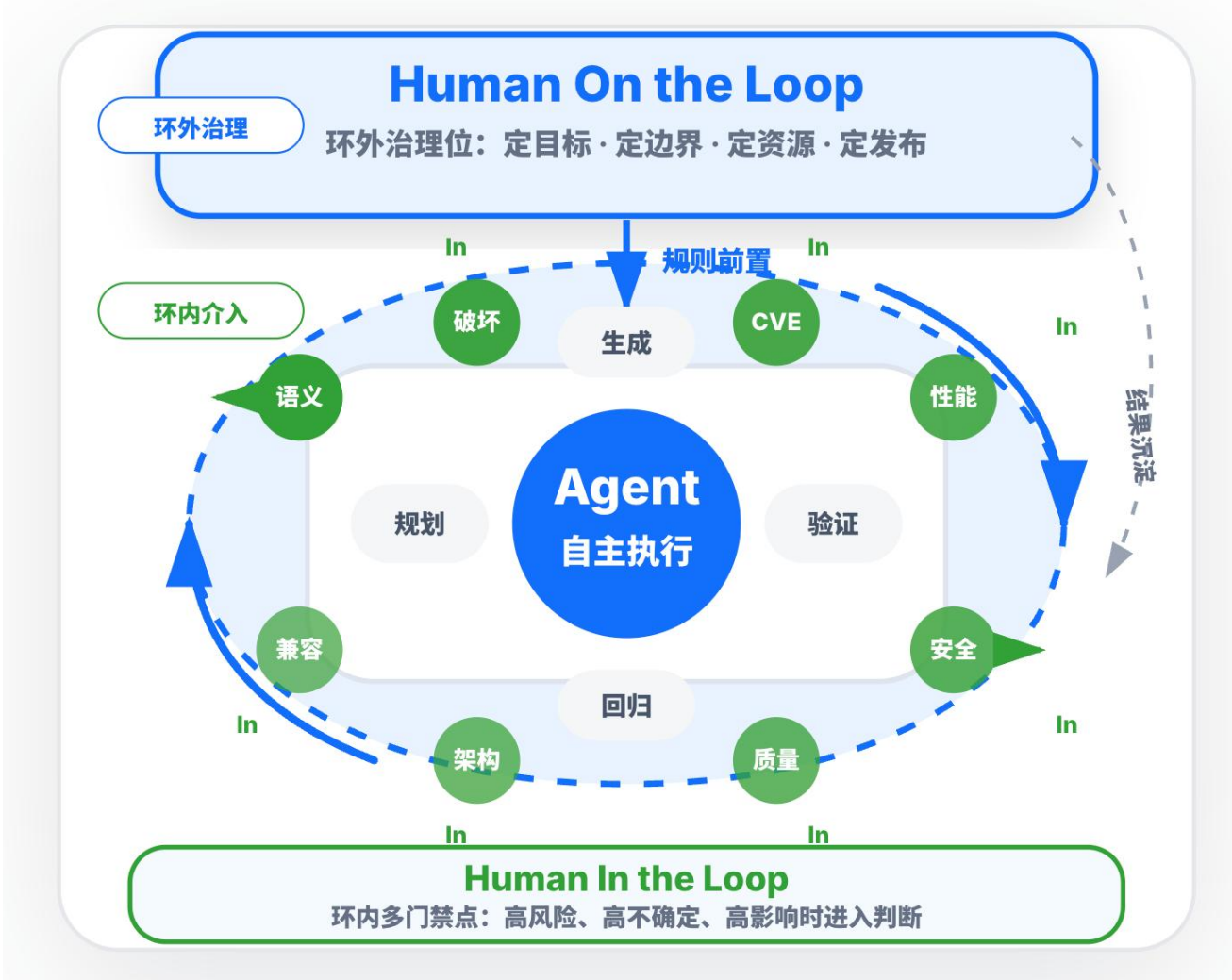
可信：做判断、做仲裁、做背书。

规则前置 | Agent 执行 | 风险触发 | 人审裁决 | 资产沉淀

Human IO Loop

IO 不是输入输出的简单缩写，而是两种人的介入方式：人在环上做治理，人在环中做关键节点判断。

让 Agent 最大化自主执行，把人的注意力集中在策略、边界、质量、风险和学习沉淀上。



Agent 为核心，解放 Agent 生产力

从 Coding Agent 单兵作战，到全流程 Agent 团队

工程师：导演 + 编剧

编排工作流程，把控质量与进度，
纠正偏差；明确需求、拆解任务、
设定边界。

Agent：舞台中心

承担核心开发、辅助生成、批量测试、
日志分析和可验证执行。

ACE Harness 产品总览



代码开发只是项目的 1/5–1/3；ACE 让分析、实现、验证、修复连续运转。

案例：社区 Issue 7×24 自动分析与 Bug 修复

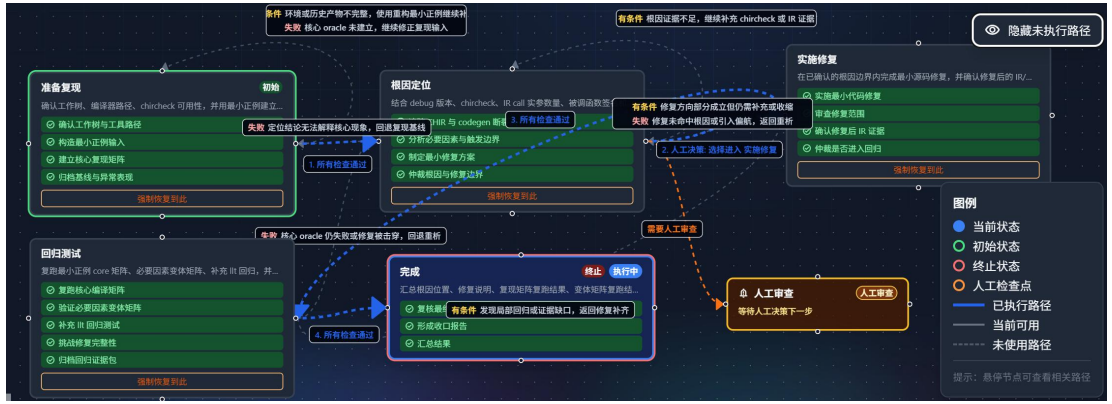
5 Agent
角色分工，状态机按步骤流转

Supervisor
构建任务环路、拆解死循环

Auto 24×7
凌晨自动运行，提前准备输入

50%+
问题分发与解决效率提升

社区 Issue 自动分析



Bug 自动修复案例



每天凌晨拉取新增 issue：提取、清洗、环境准备、分析、验证与通知；输出报告并告知库 Owner。

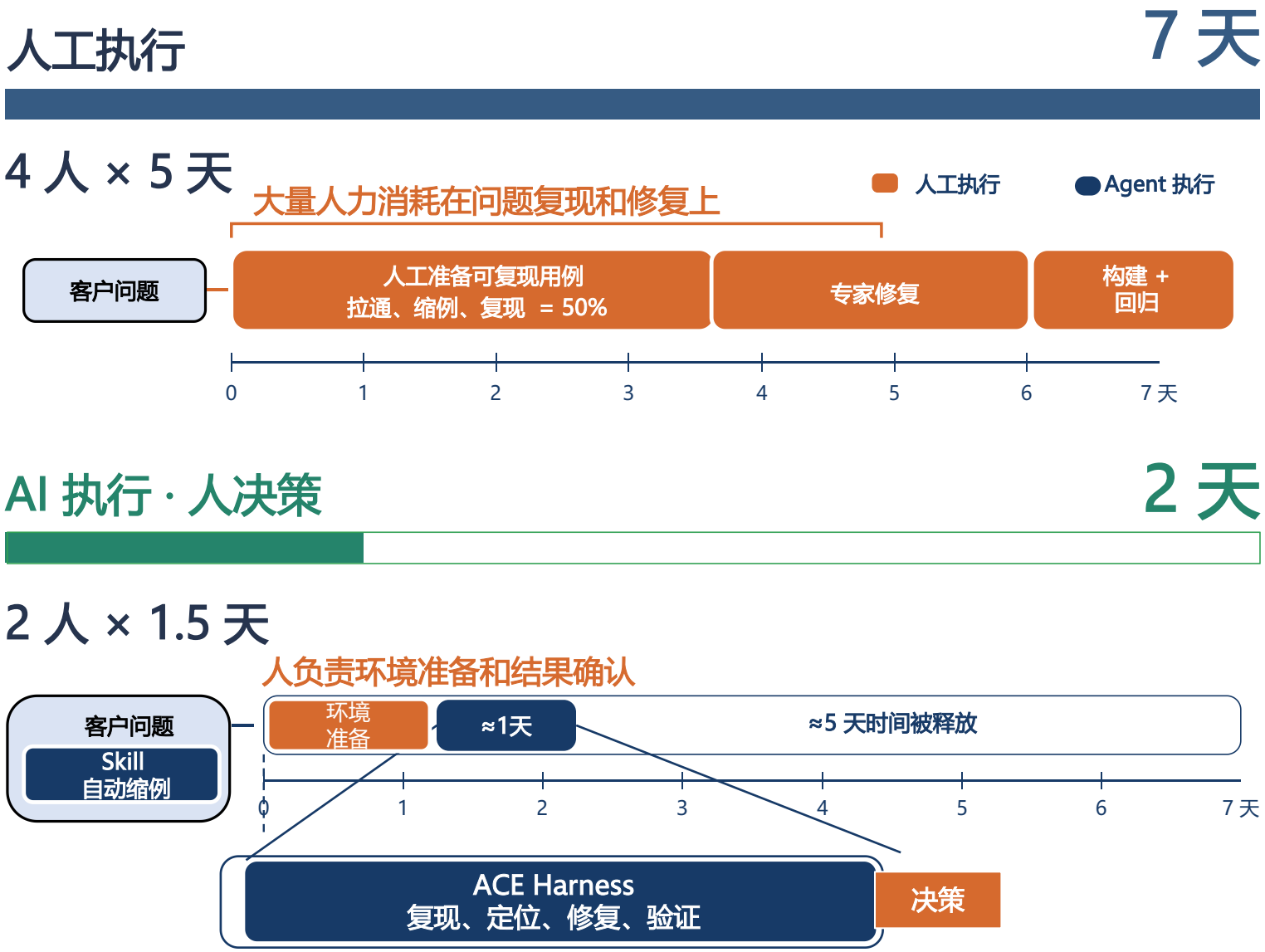
案例：仓颉编译器功能问题修复工作流

挑战

问题通常只在客户环境触发，需结合源码、IR、后端与运行时分析，并联合客户排查。复现与定位非常困难。

解决方案

Skill 支持客户初步自动缩例；基于 ACE Harness 构建复现、定位、修复、验证工作流。



In & On：库性能优化

+40%
单生产者单消费者性能

+30%
多生产者单消费者性能

1 周
从 1 个月预估压缩

旧流程卡点：公共库人力冲突

- 库团队同时支撑鸿蒙、公开、MetaERP、数通、生态等多个领域。
- 新 issue 需要等待集合数据结构专家空闲，常出现约两周等待。
- AI 缺仓颉库领域知识，开发者要持续补上下文、校验和修正。

Human On：先编组，再执行

- 组队：集合数据结构领域员工指导一名实习生，Agent 承担执行主体。
- 目标：功能正确性、性能提升、兼容性不受影响。
- 环境：编译器、Skills、Agent、库领域知识、基准与验证链路。

把“等专家亲自做”变成“专家设边界，团队带 Agent 做”

经验沉淀到知识库、基准、流程和 Agent Skills 后，同类性能问题可以更快复用。

新工作流：标准库性能 Issue 的闭环

专家不再被完整执行流程占用，而是在目标定义、方案审查、结果确认中放大判断力。

Human On | Agent | Human In

Human	Human On	Agent	Human In	Agent	Human In/On
01 Issue 入池	02 目标与编组	03 方案生成	04 双人审查	05 实现与基准	06 交付闭环
客户提交性能 issue；DTSE 判断为库问题，移交标准库团队。	专家指导实习生，前置功能、性能、兼容性目标与验证环境。	Agent 基于知识库和基准分析问题，输出候选方案并完成自研、它验。	实习生和领域专家共同审设计；有风险就要求 Agent 修正并再验证。	Agent 开发功能代码、测试用例，执行功能与性能验证，输出提升结果。	实习生确认代码、用例和性能；达标后通知 Agent 提 PR，进入检视与客户验证。

On the Loop 解决匹配
- 把领域专家、实习生、Agent 编成临时攻坚队。
- 把目标、知识、基准、兼容约束一次性前置。

Agent 解决执行
- 读基准、定位瓶颈、生成候选优化方案。
- 补代码、补用例、跑功能和性能验证。

In the Loop 解决可信
- 专家审方案和兼容性，实习生审结果。
- 不符合预期回到实现阶段，形成闭环自修复。

On：编组与前置约束　　Agent：读基准、补代码与用例、验证　　In：审方案与结果，形成闭环

从固定分工，转向基于能力标签的动态编组

ATM 是 Human IO Loop 下的作战阵型：连接任务池、人才池、Agent 池与优先级队列，匹配最合适的人机组合。

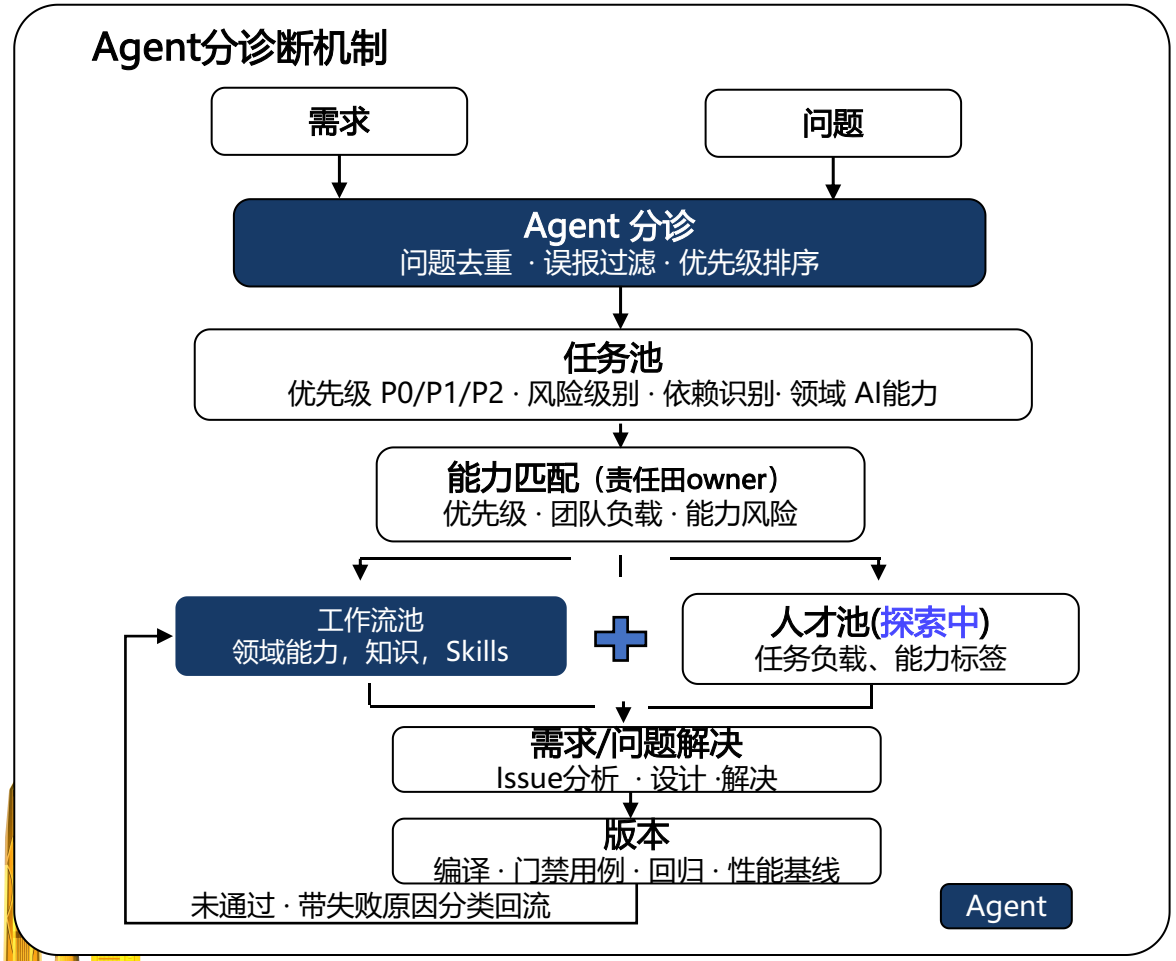


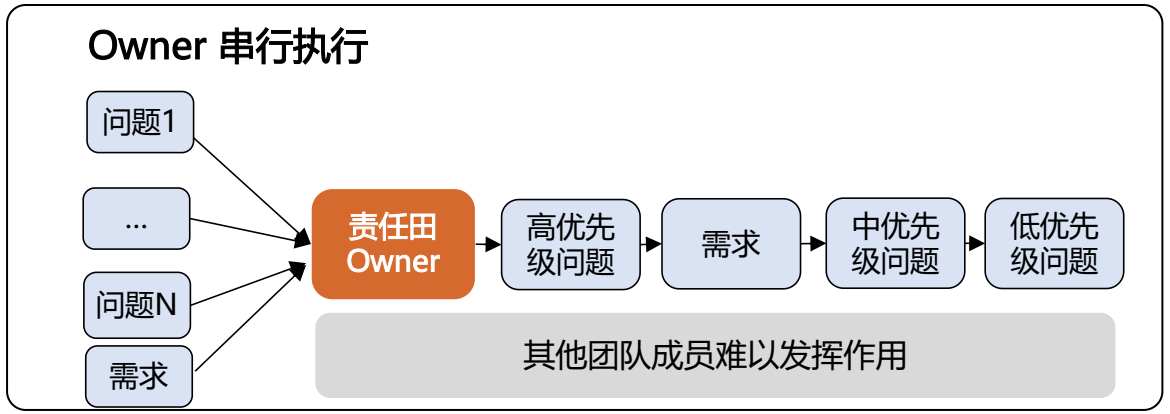
液态组织：需求攻坚小队 + Agent 工作流 + 验证门禁

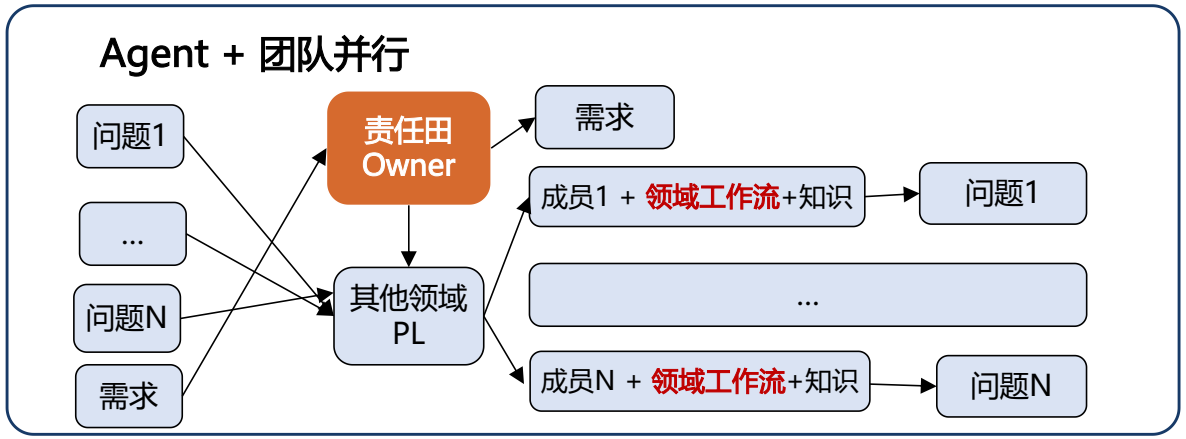
OPTIMAL ALLOCATION · FASTER TTM

基于 ATM，打破团队串行瓶颈

标准库涉及 30+ 领域，耦合较小，单个 Owner 负责 2–3 个领域。突发问题或需求撞车时，Owner 分身乏术；其他成员受领域壁垒限制，短时间内难以协助。







昇腾算子迁移：Agent 中枢编组，提升并行度

6 人
未增加团队编制

3+
并行外部需求

12.15×
FLA 最高加速比

80%+
平台达标率阶段目标

数百个 Triton 算子迁移，跨 AI 模型语义、编译器 IR、NPU 后端三层技术栈；A/B/C 客户持续输入精度、性能、亲和接口需求。固定岗位与排期造成等人、等环境、等结果，Agent 中枢按任务标签调度能力，完成后释放。

任务复杂度：不是单点优化

- GPU 平台成熟 Triton 算子生态迁移到昇腾 NPU。
- 跨 AI 模型语义、编译器 IR、NPU 后端三层技术栈。
- A/B/C 客户持续输入精度、性能、亲和接口等外部需求。

旧组织卡点：固定编制跟不上需求节奏

- 需求持续涌入，但岗位和排期固定，借调协调成本高。
- 个人能力偏单一，任务来了常常找不到完整能力组合。
- 分析等人、开发等环境、测试等结果，流程串联造成等待。

ATM 改变的是“能力调度方式”

任务先进入 Agent 中枢，Agent 识别能力缺口，再按需挂载 AI 专家、编译器骨干、工具工程骨干，完成后立即释放。

动态编组结构图：需求池 → Agent 中枢 → 能力模块

Agent 不再等人分配任务，而是持续拉取、匹配、执行、验证、修复；人只在关键节点被挂载。

AI 语义 | 编译器 IR | NPU 后端 | 工具链

需求池 Task Pool

- **A 客户 FLA 精度问题**
  标签：AI 语义 / 精度 / 方案审查
- **B 客户算子 Gap 大**
  标签：延迟 / SIMT Only / 编译器亲和接口
- **XX 平台批跑达标率**
  标签：批量分析 / Skill 命中率 / 自动统计

Agent 中枢

01 拉取任务：按优先级与风险入队
02 识别标签：语义、IR、后端、工具链
03 匹配能力：决定 0/1/2~3 人挂载策略
04 自主执行：分析、生成、验证、修复
05 沉淀资产：Pattern、Skill、日志、命中率

能力池 Capability Pool

- **AI 专家 ×2**
  语义正确性、模型算子理解、精度风险判断
- **编译器骨干 ×2**
  IR 优化、亲和接口、后端策略确认
- **工具工程骨干 ×2**
  Profiling API、批跑、日志、验证链路

标准化需求
- Agent 自主执行，0 人介入。

常规优化
- 关键方案节点挂载 1 人审核。

紧急攻坚
- 多路径并行，2~3 人短期集中。

跨域难题
- 识别能力缺口，跨组动态编组。

个人效率，最终要转化为组织能力

Human IO Loop 解决人机协作的治理边界，提升 AI 参与度；ATM 优化任务、人才与 Agent 的配置，提升并行度。两者合在一起，让团队从固定职能组织走向任务驱动的动态作战组织，提升整体效率。

人

人的价值

从“多做一步”，到判断哪一步最值得人做。

Agent

Agent 的价值

从“辅助个人”，到支撑团队连续执行。

组织

组织的价值

从“固定分工”，到“液态组织”。

Human IO Loop + ATM 的边界在哪里?

先判断任务处在 Research 还是 Development，再选择探索循环或 HIO + ATM 作战阵型。R → D：从不确定性探索，走向可验证、可复用、可规模化交付。

R -> D 从不确定性探索，走向可验证、可复用、可规模化交付。

R Research：高自由度探索，Agentic Coding

没有稳定范式，验证资产不足，过早治理会拖慢试错。

01 快速试错型 Demo 与创新头脑风暴
全新 GC 算法、新型编译架构、前沿预研；标准化治理会增加冗余。

02 高度依赖隐性经验的存量业务
经验只在专家脑中，没有文档、规则、Skills 和能力标签；Agent 难以自主适配。

03 无法搭建自动化验证体系的依赖型业务
强依赖线下环境、专属硬件、人工联合调试；缺少 Agent 自主迭代的可信反馈。

判定路径

Research
先证明路线可能成立

Evidence
形成指标、样例、失败边界

Pattern
沉淀流程、门禁、能力标签

Development

D Development：高度固定流程，HIOL + ATM

规则清楚，流程高度固定，结果能被自动验证。

01 标准化、高重复度研发流程
标准库开发、常态功能迭代、社区问题批量处理；流程固定，验收标准清晰。

02 可分层校验、可自动修复的迭代业务
上游 bug 合入、LLVM 漏洞修复、兼容性迭代；测试、合规、性能门禁能跑起来。

03 长链路、多步骤、高复杂度系统任务
编译器核心特性、NPU 算子生态迁移、大型性能优化；需要多领域能力动态聚合。

准入四问	流程是否固定?	规则是否明确?	结果能否自动验?	经验能否沉淀复用?

Research 案例：GC 预研先验证路线

Agent 先把方案、原型和数据跑出来；人再判断是否投入。

背景：CopyGC 在服务器端大内存场景下有改进空间

CopyGC 在小堆、小存活集下表现优异。但在服务器端高并发、大内存、长时运行场景下，存活对象变多，复制搬迁的账就会越来越重。

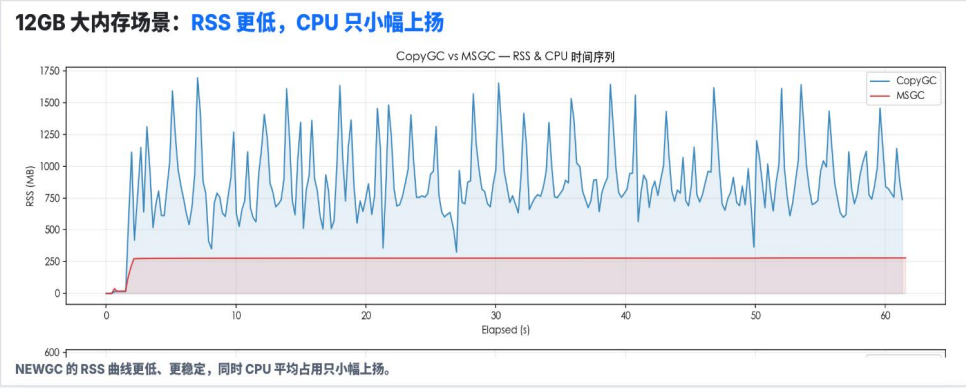
- 分配路径很快，核心是 tlRegion 指针后移；回收时再把存活对象搬到一起。
- 大存活集下，搬迁成本上来以后，GC 时间变长，RSS 也更难压下去。

投入与效果对比

Estimate → Actual

研发投入	无AI评 AI实际	4人/120人月 1人/业余4周	60%探索
Peak RSS	Copy CMS	1696MB 280MB	-83.5%
GC Time	Copy CMS	98.6ms 8.3ms	-91.6%
TPS	Copy CMS	5066 5119	+1.0%

12GB 大内存场景：RSS 更低，CPU 只小幅上扬



NEWGC 的 RSS 曲线更低、更稳定，同时 CPU 平均占用只小幅上扬。

机制对照：搬迁压缩 vs. 原位清扫复用

CopyGC

copy compact

live	free	live	copy	free	live	frag
live	live	live	free	free	free	free

优点是整理碎片很直接；代价是活对象越多，复制工作越重。

NEWGC

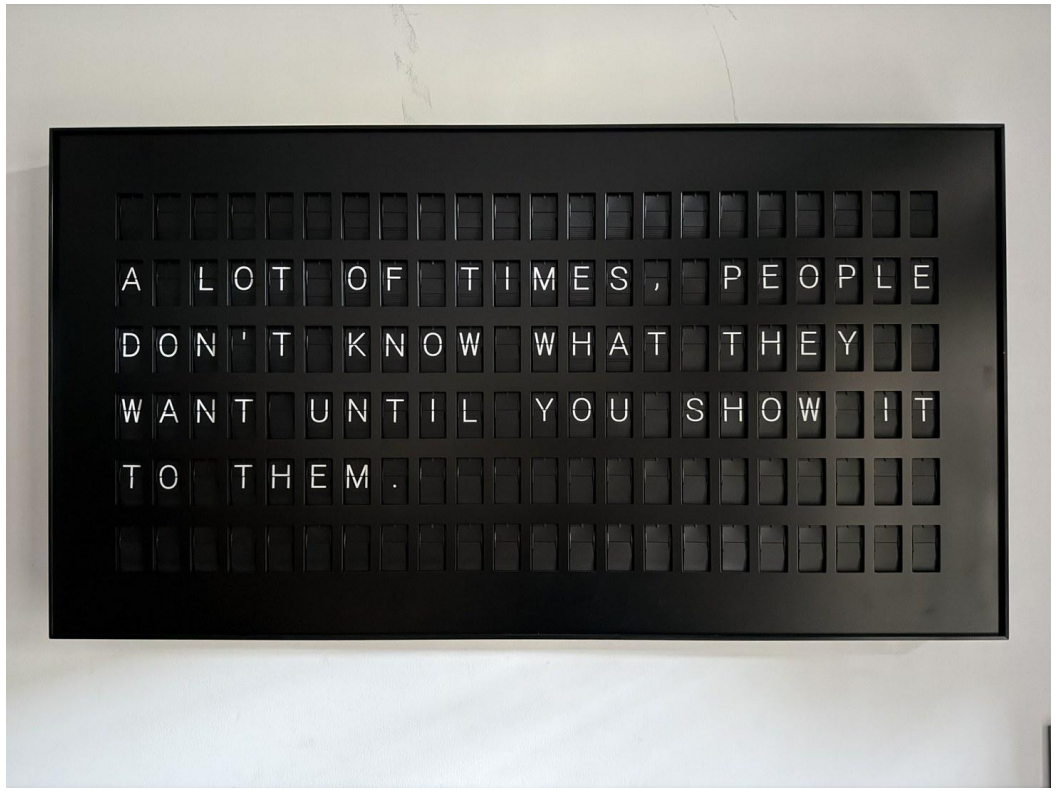
size class sweep

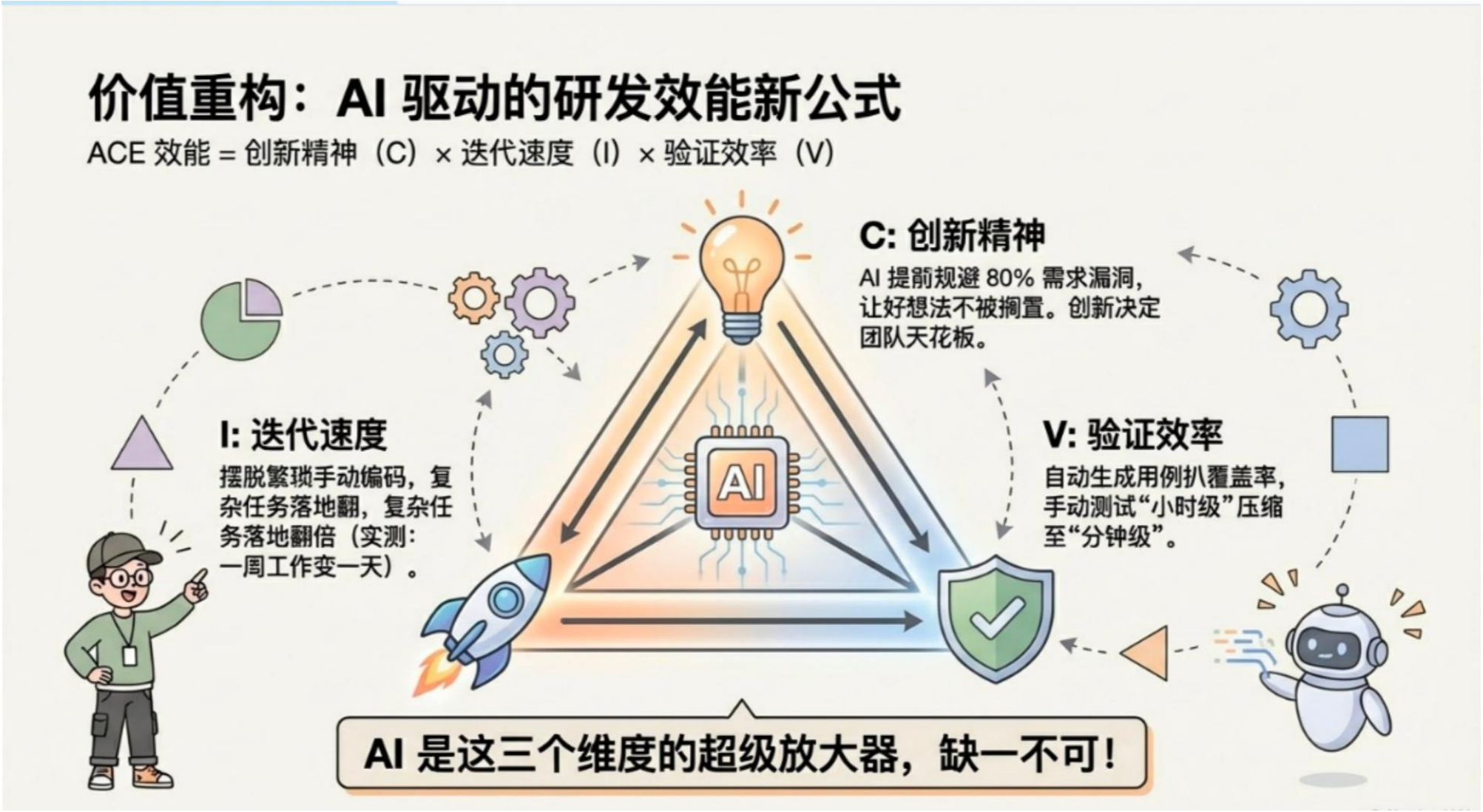
对象按大小落到 region，靠 Central Pool、lazy sweep 和 empty region 回收来复用内存。

Agent 的作用 | 细化 方案 | 编写 原型 | 构建 用例 | 性能 评估 | 复盘 沉淀

相信
> 看见

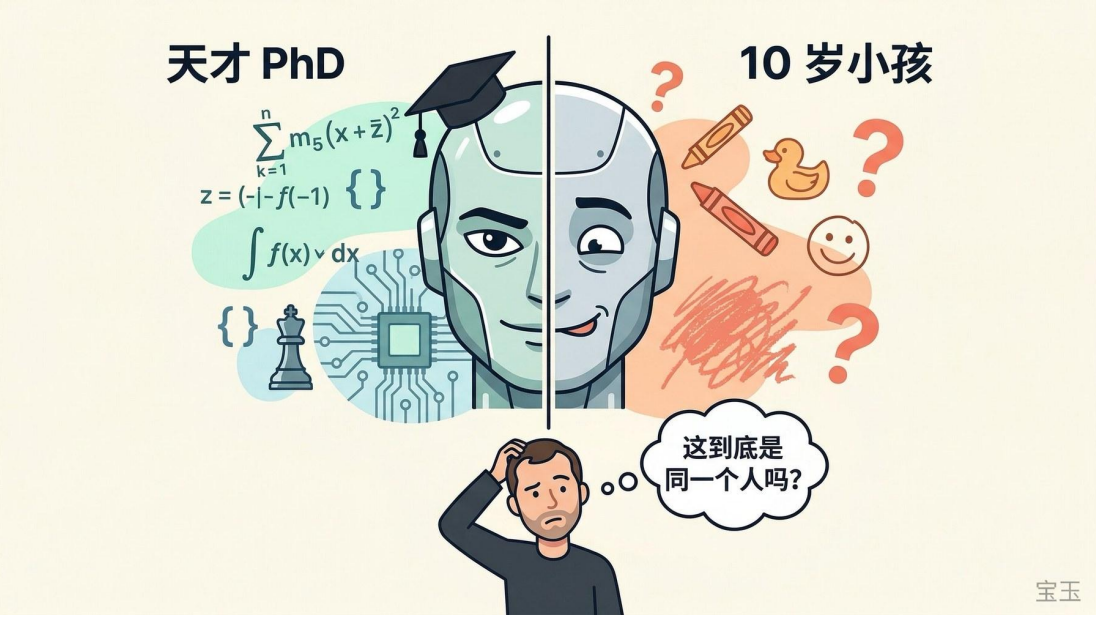
AI 研发实践常常不是“看见了才相信”，而是先相信可行，再设计可验证路径，让结果出现。




价值重构：AI 驱动的研发效能新公式
ACE 效能 = 创新精神（C）× 迭代速度（I）× 验证效率（V）
C: 创新精神
AI 提前规避 80% 需求漏洞，让好想法不被搁置。创新决定团队天花板。
I: 迭代速度
摆脱繁琐手动编码，复杂任务落地翻，复杂任务落地翻倍（实测：一周工作变一天）。
V: 验证效率
自动生成用例扒覆盖率，手动测试“小时级”压缩至“分钟级”。
AI
AI 是这三个维度的超级放大器，缺一不可！

Agent 只能改进可验证的东西

Agent 有远超人类的参差性，这是强化学习的影子；所以它只能稳定改进那些能被验证的东西。



Verification

把目标变成

测试 · 基准 · 日志
回归 · 审查标准

没有验证，Agent 只能放大不确定性。

把个人经验沉淀为可复用原则，形成团队方法论。

01

思维方式最重要

Mindset is Key
思路对了，什么都没那么难。
有 10 个高智商伙伴（AI）随时听你指挥。
好的思维是基础。

02

坚信 AI

Believe in AI
坚决不手写一行代码，不写一行文档［我的个人习惯］。
拥抱 AI 生成。

03

同路人

Like-minded Partners
寻找有同样信念的伙伴，不要试图说服所有人。
志同道合。

04

创新

Innovation
AI 是提效手段和工具。
目标是创新，创新，再创新！
目标是突破。

05

多思考，多沟通，多问为什么

给 AI 提供尽可能多的信息：你的设计、编译错误、文档、参考实现等。
信息量决定质量。

06

站在后天看明天

当下的 AI 在以周/月进步，还在加速。
用最好的模型和工具探索极限，以六个月后的 AI 能力看当下。
未来思维。

07

配合模式

把 AI 当做高智商低情商伙伴。
低情商需反复主动沟通，高智商在目标明确后极高效率。
主动沟通，明确目标。

08

AI 是个放大器

正确方向放大 100 倍，错误方向放大 100 倍。
放大你的选择。

09

做好验证

如 Unit Test 和 System Test 等。
最好的验证是标准答案，如公理定理、协议、他人既定成果。
严谨测试。

10

PEPR

Plan → Execute → Update Plan → Repeat
循环迭代。

GOSIM

GOSIM Shenzhen 2026

THANK YOU

Code · Agents · Org

